

编译原理 第九次作业

姓名：朱首赫

学号：2023K8009906029

1 练习 6.1.1

为下列表达式构建 DAG 并指出每个子表达式的值编码： $((a + b) * (a + (a + b))) - ((a + b) * (a - b))$

通过画语法树可以得到该表达式的后缀形式为： $a\ b\ +\ a\ a\ b\ +\ +\ * \ a\ b\ +\ a\ b\ -\ * \ -$ ，对后缀表达式按照从左到右的顺序依次扫描符号并创建 DAG 1 及其值编码表 1。

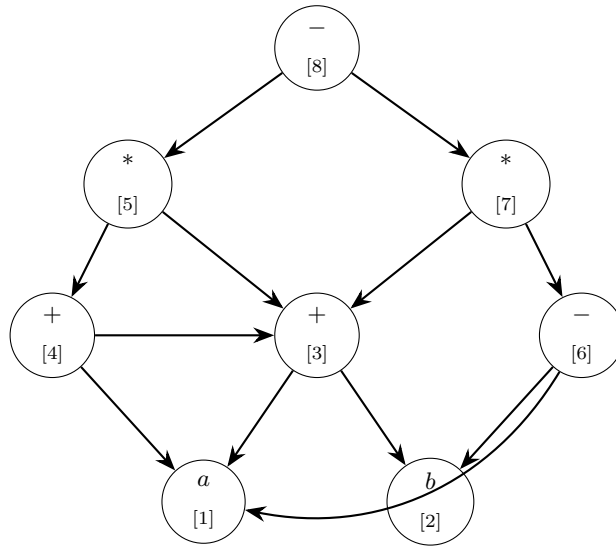


图 1: 表达式的 DAG 结构图

表 1: 值编码表

值编码 (Value Number)	操作符 / 叶节点	左孩子编码	右孩子编码
1	a	-	-
2	b	-	-
3	+	1	2
4	+	1	3
5	*	3	4
6	-	1	2
7	*	3	6
8	-	5	7

各值编码所对应的具体子表达式展开如下：

- [1]: a
- [2]: b

- [3]: $a + b$
- [4]: $a + (a + b)$
- [5]: $(a + b) * (a + (a + b))$
- [6]: $a - b$
- [7]: $(a + b) * (a - b)$
- [8]: $((a + b) * (a + (a + b))) - ((a + b) * (a - b))$

2 练习 6.1.2

将语句 $a = (b + c) * (- (b + c))$ 翻译为:

1. 抽象语法树
 2. 四元式序列
 3. 三元式序列
 4. 间接三元式序列
1. 抽象语法树如下:

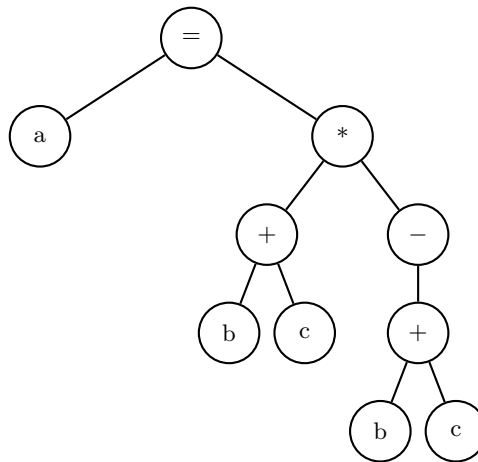


图 2: 抽象语法树

2. 四元式序列: 由语法树自底向上构建四元式序列

表 2: 四元式序列

序号	op	arg1	arg2	result	语义说明
(0)	+	b	c	t_1	$t_1 = b + c$
(1)	+	b	c	t_2	$t_2 = b + c$
(2)	uminus	t_2	—	t_3	$t_3 = -t_2$
(3)	$\times$	t_1	t_3	t_4	$t_4 = t_1 \times t_3$
(4)	=	t_4	—	a	$a = t_4$

3. 三元式序列:
4. 间接三元式序列:

表 3: 三元式中间代码序列

序号	op	arg1	arg2	语义说明
(0)	+	b	c	计算第一个 $b + c$
(1)	+	b	c	计算第二个 $b + c$
(2)	uminus	(1)	—	对第 (1) 步的结果取负
(3)	×	(0)	(2)	将第 (0) 步与第 (2) 步结果相乘
(4)	=	a	(3)	将第 (3) 步结果赋值给变量 a

表 4: 间接三元式表示法

(a) 执行顺序表 (Execution Order)

位置	三元式指针
0	(0)
1	(1)
2	(2)
3	(3)
4	(4)

(b) 三元式实体表 (Triples Table)

标识	op	arg1	arg2
(0)	+	b	c
(1)	+	b	c
(2)	uminus	(1)	—
(3)	×	(0)	(2)
(4)	=	a	(3)

3 练习 6.1.3

确定下列声明序列中各个标识符的类型和相对地址。

```
1 float x;
2 record { float x; float y; } p;
3 record {
4     record { int tag; float x; } m;
5     record { int idx; float y;} n;
6 } q;
```

表 5: 标识符类型与地址分配表 (int=4, float=4)

标识符	类型表达式 (Type Expression)	大小	内部偏移	全局偏移
x	float	4	—	0
p	record(x: float, y: float)	8	—	4
p.x	float	4	0	4
p.y	float	4	4	8
q	record(m: record(...), n: record(...))	16	—	12
q.m	record(tag: int, x: float)	8	0	12
q.m.tag	int	4	0	12
q.m.x	float	4	4	16
q.n	record(idx: int, y: float)	8	8	20
q.n.idx	int	4	0	20
q.n.y	float	4	4	24

4 练习 6.1.4

考虑龙书图 6-22 的翻译方案，翻译下列赋值语句并给出注释语法分析树。 $x = a[b[i][j]][c[k]]$;

根据图 6-22 的语义动作，非终结符具有以下综合属性：

- $E.addr$: 存储表达式值的临时变量或名字。
- $L.addr$: 存储数组元素偏移量的临时变量。
- $L.array$: 指向符号表中数组条目的指针（通过 $L.array.base$ 获取数组首地址）。
- $L.type$: 当前子数组的类型，用于获取当前维度的元素宽度 ($L.type.width$)。

对于赋值语句 $x = a[b[i][j]][c[k]]$ ；，按照自底向上的规约顺序，生成的三地址代码序列如下：

表 6: 三地址代码生成序列

步骤	对应语法规约	产生的三地址代码
1	$L \rightarrow id[E]$ (规约 $b[i]$)	$t_1 = i \times b.type.elem.width$
2	$L \rightarrow L_1[E]$ (规约 $b[i][j]$)	$t_2 = j \times b.type.elem.elem.width$ $t_3 = t_1 + t_2$
3	$E \rightarrow L$ (表达式中的数组引用)	$t_4 = b[t_3]$
4	$L \rightarrow id[E]$ (规约 $a[b[i][j]]$)	$t_5 = t_4 \times a.type.elem.width$
5	$L \rightarrow id[E]$ (规约 $c[k]$)	$t_6 = k \times c.type.elem.width$
6	$E \rightarrow L$ (表达式中的数组引用)	$t_7 = c[t_6]$
7	$L \rightarrow L_1[E]$ (规约整个右侧数组引用)	$t_8 = t_7 \times a.type.elem.elem.width$ $t_9 = t_5 + t_8$
8	$E \rightarrow L$ (右侧整体规约为 E)	$t_{10} = a[t_9]$
9	$S \rightarrow id = E$; (赋值语句)	$x = t_{10}$

以下是完整的注释语法分析树。每个非终结符节点均标明了其合成的属性值（如 *addr*, *array*, *type*）:

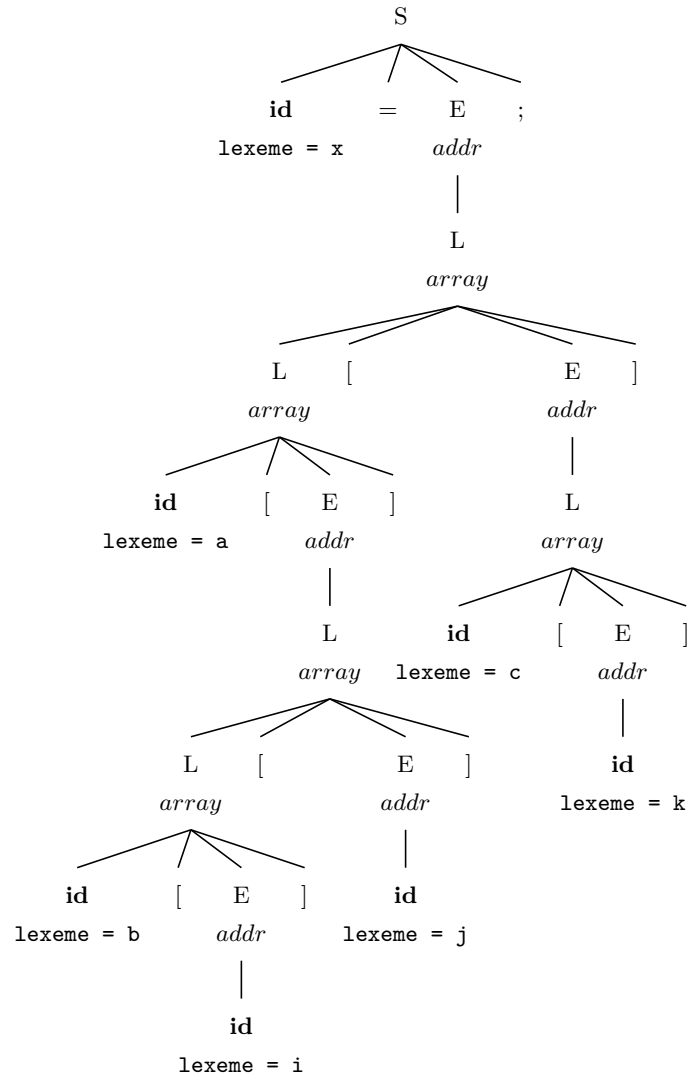


图 3: 注释语法分析树